

ensure that every user has a truly unique username, not just a variation on the capitalization of another person's username. When someone submits a new username, that new username is converted to lowercase and compared to the lowercase versions of all existing usernames. During this check, a username like 'John' will be rejected if any variation of 'john' is already in use.

Checking for Inequality

When you want to determine whether two values are not equal, you can use the *inequality operator* (`!=`). Let's use another if statement to examine how to use the inequality operator. We'll store a requested pizza topping in a variable and then print a message if the person did not order anchovies:

```
toppings.py requested_topping = 'mushrooms'

if requested_topping != 'anchovies':
    print("Hold the anchovies!")
```

This code compares the value of `requested_topping` to the value `'anchovies'`. If these two values do not match, Python returns `True` and executes the code following the if statement. If the two values match, Python returns `False` and does not run the code following the if statement.

Because the value of `requested_topping` is not `'anchovies'`, the `print()` function is executed:

```
Hold the anchovies!
```

Most of the conditional expressions you write will test for equality, but sometimes you'll find it more efficient to test for inequality.

Numerical Comparisons

Testing numerical values is pretty straightforward. For example, the following code checks whether a person is 18 years old:

```
>>> age = 18
>>> age == 18
True
```

You can also test to see if two numbers are not equal. For example, the following code prints a message if the given answer is not correct:

```
magic _number.py answer = 17
if answer != 42:
    print("That is not the correct answer. Please try again!")
```

The conditional test passes, because the value of `answer` (17) is not equal to 42. Because the test passes, the indented code block is executed:

```
That is not the correct answer. Please try again!
```
